# Harish K S – Milestone Assessment 03 – iOS with SwiftUI && Core Java

## Fitness Application

**Project Name: Fitness**

**Core Data Model (.xcdatamodeld file)**

Adding an entity called **Workout** with attributes:

- exerciseName: String
- duration: Double
- caloriesBurned: Double

**PersistentController**

```
import CoreData

struct PersistenceController {
    static let shared = PersistenceController()
    let container: NSPersistentContainer

    init(inMemory: Bool = false) {
        container = NSPersistentContainer(name: "Fitness")

        if inMemory {
            container.persistentStoreDescriptions.first?.url = URL(fileURLWithPath: "/dev/null")

        }

        container.loadPersistentStores {_ , error in
            if let error = error as NSError? {
                fatalError("Unresolved Error: \(error), \(error.userInfo)")
            }
        }
    }


    func saveContext() {
        let context = container.viewContext
        if context.hasChanges {
            try? context.save()
        }
    }

}
```

**FitnessApp**

```
import SwiftUI

@main
struct FitnessApp: App {
   let persistentController = PersistenceController.shared

   var body: some Scene {
      WindowGroup {
         WorkoutListView()
            .environment(\.managedObjectContext, persistentController.container.viewContext)
      }
   }
}
```

**WorkoutListView**

```
import SwiftUI

struct WorkoutListView: View {
   @Environment(\.managedObjectContext) private var viewContext
   @FetchRequest(
      entity: Workout.entity(),
      sortDescriptors: []
   )private var workouts: FetchedResults<Workout>

   @State private var showAddWorkoutView = false
   @State private var editWorkout: Workout?

   var body: some View {
      NavigationView {
         VStack {
            List {
               ForEach(workouts) { workout in
                  NavigationLink(destination: EditWorkoutView(workout: workout)) {
                     VStack(alignment: .leading, spacing: 5) {
                        Text(workout.exerciseName ?? "No title")
                           .font(.title2)
                           .padding(.vertical, 5)

                        Text("Calories Burned: \(workout.caloriesBurned, specifier: "%.2f") cal")
                           .font(.title3)
                           .padding(.vertical, 5)

                        Text("Duration: \(workout.duration, specifier: "%.2f") min")
                           .font(.title3)
                           .padding(.vertical, 5)
                     }
```

```
                }
                .buttonStyle(PlainButtonStyle())
                .swipeActions {
                    Button(role: .destructive) {
                        deleteWorkout(workout)
                    } label: {
                        Label("Delete", systemImage: "trash")
                    }
                }
            }
        }
        .navigationTitle("Workouts List")
        .toolbar {
            ToolbarItem(placement: .navigationBarTrailing) {
                Button(action: { showAddWorkoutView = true }) {
                    Image(systemName: "plus")
                }
            }
        }
        .sheet(isPresented: $showAddWorkoutView) {
            AddWorkoutView()
        }
    }
  }

  private func deleteWorkout(_ workout : Workout) {
    viewContext.delete(workout)

    DispatchQueue.main.async {
        do {
            try viewContext.save()
        } catch {
            print("Error deleting workout: \(error.localizedDescription)")
        }
    }
  }

}

struct WorkoutListView_Previews: PreviewProvider {
  static var previews: some View {
    WorkoutListView()
  }
}
```

**AddWorkoutView**

```
import SwiftUI

struct AddWorkoutView: View {
    @Environment(\.managedObjectContext) private var viewContext
    @Environment(\.dismiss) private var dismiss

    @State private var exerciseName = ""
    @State private var caloriesBurned = ""
    @State private var duration = ""

    var body: some View {
        NavigationView {
            Form {
                Section(header: Text("Exercise Details")) {
                    TextField("Enter Exercise", text: $exerciseName)
                        .textFieldStyle(RoundedBorderTextFieldStyle())

                    TextField("Enter Calories Burned", text: $caloriesBurned)
                        .textFieldStyle(RoundedBorderTextFieldStyle())

                    TextField("Enter Duration", text: $duration)
                        .textFieldStyle(RoundedBorderTextFieldStyle())
                }
            }
            .navigationTitle("Add Exercise")
            .navigationBarItems(
                leading: Button("Cancel") { dismiss() },
                trailing: Button("Save") {
                    addExercise()
                    dismiss()
                }
                    .disabled(exerciseName.isEmpty || caloriesBurned.isEmpty || duration.isEmpty)
            )
        }
    }

    private func addExercise() {
        let newExercise = Workout(context: viewContext)
        newExercise.exerciseName = exerciseName
        newExercise.caloriesBurned = Double(caloriesBurned) ?? 0.0
        newExercise.duration = Double(duration) ?? 0.0

        DispatchQueue.main.async {
            do {
                try viewContext.save()
            } catch {
                print("Error adding workout: \(error.localizedDescription)")
            }
```

```
        }
    }
}

struct AddWorkoutView_Previews: PreviewProvider {
    static var previews: some View {
        AddWorkoutView()
    }
}
```

**EditWorkoutView**

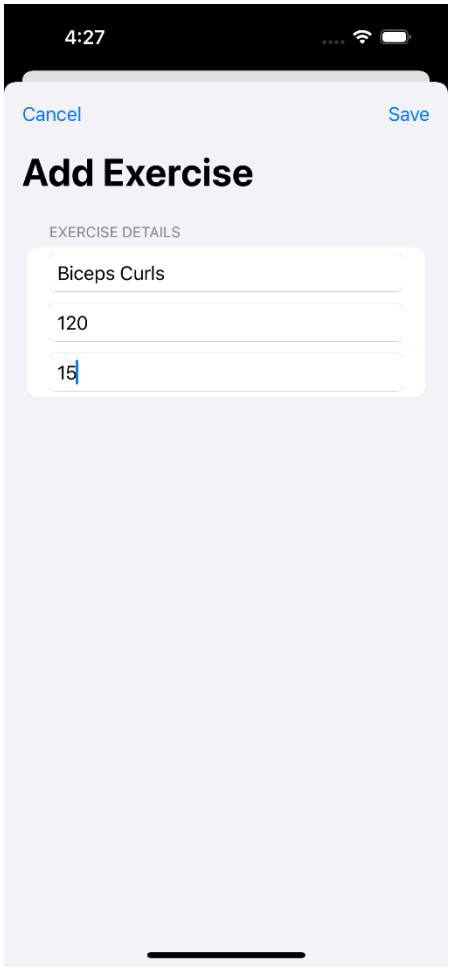
```
import SwiftUI

struct EditWorkoutView: View {
    @Environment(\.managedObjectContext) private var viewContext
    @Environment(\.dismiss) private var dismiss

    @ObservedObject var workout: Workout

    @State private var exerciseName = ""
    @State private var caloriesBurned = ""
    @State private var duration = ""

    var body: some View {
        NavigationView {
            Form {
                Section(header: Text("Exercise Details")) {
                    TextField("Enter Exercise", text: $exerciseName)
                        .textFieldStyle(RoundedBorderTextFieldStyle())

                    TextField("Enter Calories Burned", text: $caloriesBurned)
                        .textFieldStyle(RoundedBorderTextFieldStyle())

                    TextField("Enter Duration", text: $duration)
                        .textFieldStyle(RoundedBorderTextFieldStyle())
                }
            }
            .navigationTitle("Edit Exercise")
            .navigationBarItems(
                leading: Button("Cancel") { dismiss() },
                trailing: Button("Save") {
                    updateExercise()
                    dismiss()
                }
                    .disabled(exerciseName.isEmpty || caloriesBurned.isEmpty || duration.isEmpty)
            )
            .onAppear {
                exerciseName = workout.exerciseName ?? ""
```

```
            caloriesBurned = String(format: "%.2f", workout.caloriesBurned)
            duration = String(format: "%.2f", workout.duration)
        }
    }
}

private func updateExercise() {
    workout.exerciseName = exerciseName
    workout.caloriesBurned = Double(caloriesBurned) ?? 0.0
    workout.duration = Double(duration) ?? 0.0

    DispatchQueue.main.async {
        do {
            try viewContext.save()
        } catch {
            print("Error adding workout: \(error.localizedDescription)")
        }
    }
}
}
```

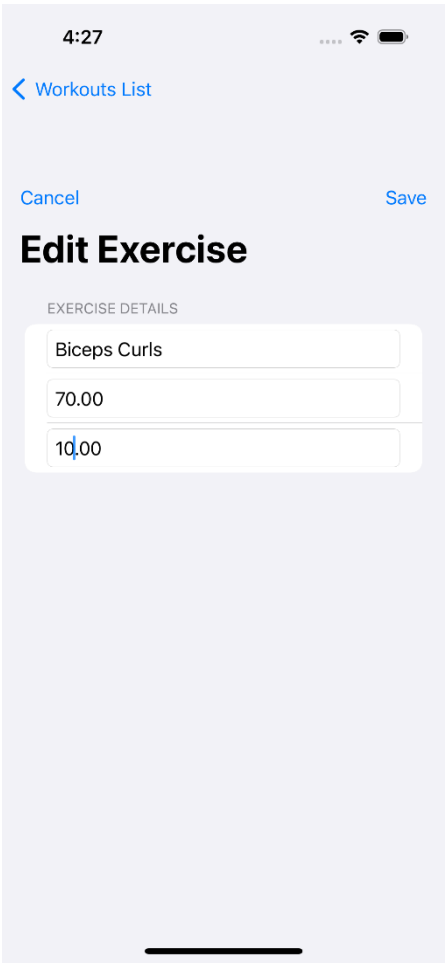
**Output:**

01_WorkoutListView

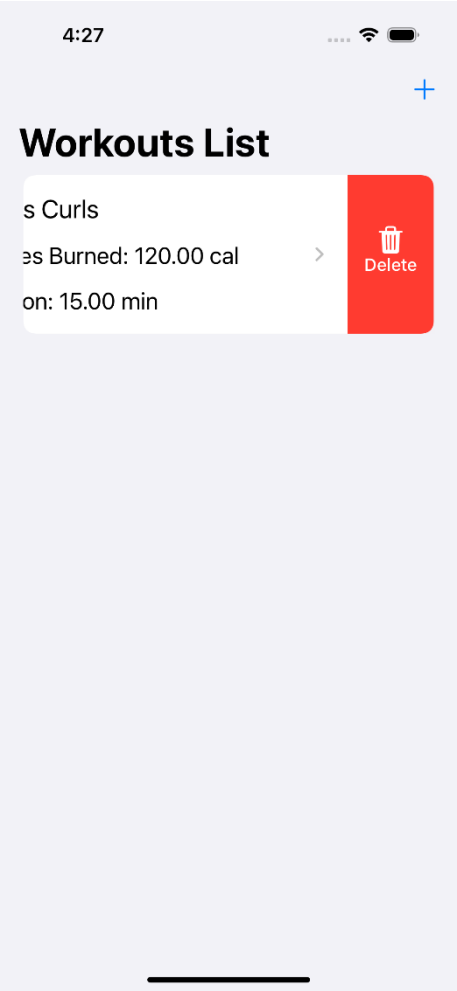
02_AddWorkoutView



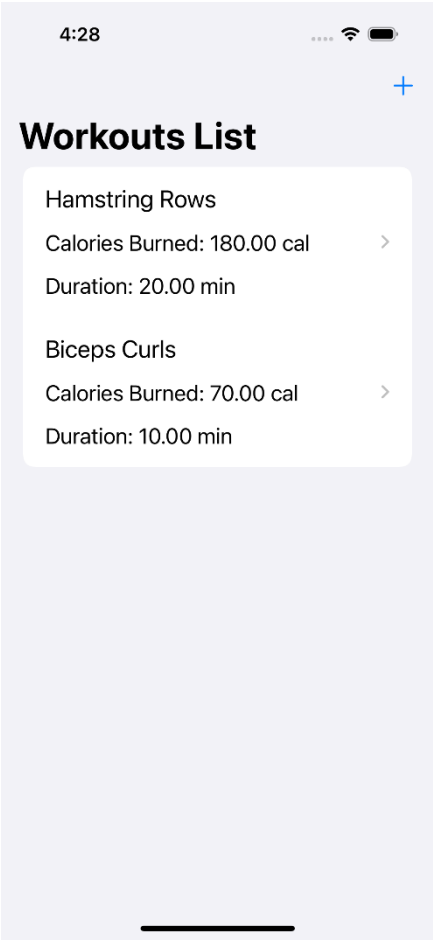

03_EditWorkoutView



04_DeleteWorkout

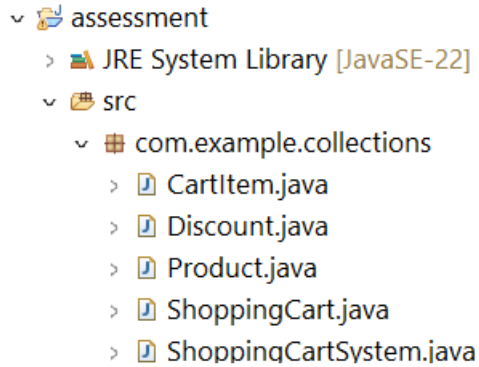



05_WorkoutListView



# Core Java – Shooping Cart System

## Package Structure:




**Product.java**

```
package com.example.collections;

public class Product {
	private int productId;
	private String name;
	private double price;
	private String category;

	public Product(int productId, String name, double price, String category) {
		this.productId = productId;
		this.name = name;
		this.price = price;
		this.category = category;
	}

	public int getProductId() {
		return productId;
	}

	public String getName() {
		return name;
	}

	public double getPrice() {
		return price;
	}

	public String getCategory() {
		return category;
	}

	@Override
	public String toString() {
		return "Product ID: " + productId + ", Name: " + name + ", Price: " + price + ", category: "
+ category;
	}

}
```

**CartItem.java**

```
package com.example.collections;

public class CartItem {
	private Product product;
	private int quantity;

	public CartItem(Product product, int quantity) {
		this.product = product;
		this.quantity = quantity;
	}

	public Product getProduct() {
		return product;
	}

	public int getQuantity() {
		return quantity;
	}

	public void setQuantity(int quantity) {
		this.quantity = quantity;
	}

}
```

**Discount.java**

```
package com.example.collections;

public class Discount {
	private double discountPercentage;
	private double cartValue;

	public Discount(double discountPercentage, double cartValue) {
		this.discountPercentage = discountPercentage;
		this.cartValue = cartValue;
	}

	public double getDiscountPercentage() {
		return discountPercentage;
	}

	public double getCartValue() {
		return cartValue;
	}

}
```

**ShoppingCart.java**

```
package com.example.collections;

import java.util.ArrayList;
import java.util.List;

public class ShoppingCart {
	private List<CartItem> cartItems;
	private Discount discount;

	public ShoppingCart() {
		this.cartItems = new ArrayList<>();
	}

	public void addProduct(Product product, int quantity) {
		for(CartItem item : cartItems) {
			if(item.getProduct().getProductId() == product.getProductId()) {
				item.setQuantity(item.getQuantity() + quantity);
				return;
			}
		}
		cartItems.add(new CartItem(product, quantity));
	}

	public void removeProduct(int productId) {
		cartItems.removeIf(item -> item.getProduct().getProductId() == productId);
	}

	public void updateProductQuantity(int productId, int quantity) {
		for(CartItem item : cartItems) {
			if(item.getProduct().getProductId() == productId) {
				item.setQuantity(quantity);
				return;
			}
		}
	}

	public void applyDiscount(Discount discount) {
		this.discount = discount;
	}

	public double calculateTotal() {
		double total = 0;
		for(CartItem item : cartItems) {
			total += item.getProduct().getPrice() * item.getQuantity();
		}
		if(discount != null && total >= discount.getCartValue()) {
			total -= (total * discount.getDiscountPercentage() / 100);
		}
		return total;
	}

	public void displayProducts() {
		System.out.println("Products in the cart:");
		for(CartItem item : cartItems) {
			System.out.println(item.getProduct() + ", Quantity: " + item.getQuantity());
```

```
        }
    }

}
```

**ShoppingCartSystem.java**

```
package com.example.collections;

public class ShoppingCartSystem {

    public static void main(String[] args) {
        Product product1 = new Product(1, "Television", 55000, "Electronics");
        Product product2 = new Product(2, "IPhone-14Pro", 96000, "Electronics");
        Product product3 = new Product(3, "HarryPotterBooks", 12500, "Books");

        ShoppingCart cart = new ShoppingCart();
        cart.addProduct(product1, 2);
        cart.addProduct(product2, 2);
        cart.addProduct(product3, 1);

        cart.displayProducts();

        System.out.println("\nTotal Value of Cart Items before Discount: " + cart.calculateTotal());

        Discount discount = new Discount(10, 5000);
        cart.applyDiscount(discount);

        System.out.println("\n10 % discount for products above 5000");

        System.out.println("\nTotal Value of Cart Items after Discount: " + cart.calculateTotal());
    }

}
```

**Output:**

```
Products in the cart:
Product ID: 1, Name: Television, Price: 55000.0, category: Electronics, Quantity: 2
Product ID: 2, Name: IPhone-14Pro, Price: 96000.0, category: Electronics, Quantity: 2
Product ID: 3, Name: HarryPotterBooks, Price: 12500.0, category: Books, Quantity: 1

Total Value of Cart Items before Discount: 314500.0

10 % discount for products above 5000

Total Value of Cart Items after Discount: 283050.0
```